

Fuzzy Logic Tipping System

This notebook implements a fuzzy logic tipping system using three different types of membership functions: Triangular, Trapezoidal, and Gaussian. It compares their outputs for various test cases.

```
In [1]: # Import necessary libraries
import matplotlib.pyplot as plt
import numpy as np
```

Base Fuzzy Set Class

This class defines the basic properties and methods for a fuzzy set, including initialization, setting membership values, getting membership values for a specific input, and plotting.

```
In [2]: class FuzzySet:
    def __init__(self, name, x_min, x_max, resolution=100):
        self.name = name
        self.x_min = x_min
        self.x_max = x_max
        self.x = np.linspace(x_min, x_max, resolution)
        self.membership = np.zeros_like(self.x)

    def set_membership(self, membership_values):
        if len(membership_values) != len(self.x):
            try:
                # Interpolate if lengths don't match
                self.membership = np.interp(self.x, np.linspace(self.x_min,
                                                                self.x_max, len(membership_values)),
                                             membership_values)
            except Exception as e:
                raise ValueError(f"Length mismatch: {len(membership_values)}")
        else:
            self.membership = np.array(membership_values)

    def get_membership(self, val):
        return np.interp(val, self.x, self.membership)

    def plot(self, ax=None):
        if ax is None:
            fig, ax = plt.subplots()
        ax.plot(self.x, self.membership, label=self.name)
        return ax
```

Triangular Fuzzy Set

This class implements a triangular membership function, defined by three points: `a` (left foot), `b` (peak), and `c` (right foot).

```
In [3]: class TriangularFuzzySet(FuzzySet):
    def __init__(self, name, x_min, x_max, a, b, c, resolution=100):
        super().__init__(name, x_min, x_max, resolution)
        self.a = a
        self.b = b
        self.c = c
        self._calculate_membership()

    def _calculate_membership(self):
        y = np.zeros_like(self.x)
        mask_left = (self.x >= self.a) & (self.x <= self.b)
        mask_right = (self.x >= self.b) & (self.x <= self.c)

        if self.b != self.a:
            y[mask_left] = (self.x[mask_left] - self.a) / (self.b - self.a)
        else:
            y[mask_left] = 1.0 # Handle vertical edge case

        if self.c != self.b:
            y[mask_right] = (self.c - self.x[mask_right]) / (self.c - self.b)
        else:
            y[mask_right] = 1.0

        self.membership = np.clip(y, 0, 1)
```

Trapezoidal Fuzzy Set

This class implements a trapezoidal membership function, defined by four points: **a** (left foot), **b** (left shoulder), **c** (right shoulder), and **d** (right foot).

```
In [4]: class TrapezoidalFuzzySet(FuzzySet):
    def __init__(self, name, x_min, x_max, a, b, c, d, resolution=100):
        super().__init__(name, x_min, x_max, resolution)
        self.a = a
        self.b = b
        self.c = c
        self.d = d
        self._calculate_membership()

    def _calculate_membership(self):
        y = np.zeros_like(self.x)
        y[(self.x >= self.b) & (self.x <= self.c)] = 1.0

        mask_left = (self.x >= self.a) & (self.x < self.b)
        if self.b != self.a:
            y[mask_left] = (self.x[mask_left] - self.a) / (self.b - self.a)

        mask_right = (self.x > self.c) & (self.x <= self.d)
        if self.d != self.c:
            y[mask_right] = (self.d - self.x[mask_right]) / (self.d - self.c)

        self.membership = np.clip(y, 0, 1)
```

Gaussian Fuzzy Set

This class implements a Gaussian membership function, defined by a `mean` and a standard deviation (`sigma`).

```
In [5]: class GaussianFuzzySet(FuzzySet):
    def __init__(self, name, x_min, x_max, mean, sigma, resolution=100):
        super().__init__(name, x_min, x_max, resolution)
        self.mean = mean
        self.sigma = sigma
        self._calculate_membership()

    def _calculate_membership(self):
        y = np.exp(-0.5 * ((self.x - self.mean) / self.sigma) ** 2)
        self.membership = np.clip(y, 0, 1)
```

Defuzzification

The `defuzzify_centroid` function calculates the crisp output value by finding the center of gravity (centroid) of the aggregated fuzzy set.

```
In [6]: def defuzzify_centroid(x, y):
    if np.sum(y) == 0:
        return 0
    return np.sum(x * y) / np.sum(y)
```

Tipping Calculation (Triangular)

This function calculates the tip percentage based on service and food quality ratings using **Triangular** membership functions.

```
In [7]: def calculate_tip_triangular(input_service, input_food):
    """Fuzzy inference system using TRIANGULAR membership functions"""
    service_range = (0, 10)
    food_range = (0, 10)
    tip_range = (0, 25)

    # Service: Poor, Good, Excellent (Triangular)
    service_poor = TriangularFuzzySet("Poor", *service_range, 0, 0, 5)
    service_good = TriangularFuzzySet("Good", *service_range, 0, 5, 10)
    service_excellent = TriangularFuzzySet("Excellent", *service_range, 5, 10, 10)

    # Food: Rancid, Delicious (Triangular)
    food_rancid = TriangularFuzzySet("Rancid", *food_range, 0, 0, 4)
    food_delicious = TriangularFuzzySet("Delicious", *food_range, 6, 10, 10)

    # Tip: Cheap, Average, Generous (Triangular)
    tip_cheap = TriangularFuzzySet("Cheap", *tip_range, 0, 5, 10)
    tip_average = TriangularFuzzySet("Average", *tip_range, 10, 15, 20)
```

```

tip_generous = TriangularFuzzySet("Generous", *tip_range, 15, 25, 25)

# Rule Evaluation
mu_service_poor = service_poor.get_membership(input_service)
mu_food_rancid = food_rancid.get_membership(input_food)
rule1_strength = max(mu_service_poor, mu_food_rancid)

mu_service_good = service_good.get_membership(input_service)
rule2_strength = mu_service_good

mu_service_excellent = service_excellent.get_membership(input_service)
mu_food_delicious = food_delicious.get_membership(input_food)
rule3_strength = max(mu_service_excellent, mu_food_delicious)

# Activation
tip_cheap_activation = np.minimum(rule1_strength, tip_cheap.membership)
tip_average_activation = np.minimum(rule2_strength, tip_average.membership)
tip_generous_activation = np.minimum(rule3_strength, tip_generous.membership)

# Aggregation
aggregated_output = np.maximum(tip_cheap_activation,
                                np.maximum(tip_average_activation, tip_generous_activation))

# Defuzzification
output_tip = defuzzify_centroid(tip_cheap.x, aggregated_output)

return output_tip, {
    'rule1': rule1_strength,
    'rule2': rule2_strength,
    'rule3': rule3_strength
}

```

Tipping Calculation (Trapezoidal)

This function calculates the tip percentage using **Trapezoidal** membership functions, offering a different shape for fuzzy sets.

```

In [8]: def calculate_tip_trapezoidal(input_service, input_food):
        """Fuzzy inference system using TRAPEZOIDAL membership functions"""
        service_range = (0, 10)
        food_range = (0, 10)
        tip_range = (0, 25)

        # Service: Poor, Good, Excellent (Trapezoidal)
        service_poor = TrapezoidalFuzzySet("Poor", *service_range, 0, 0, 2, 5)
        service_good = TrapezoidalFuzzySet("Good", *service_range, 3, 5, 7, 9)
        service_excellent = TrapezoidalFuzzySet("Excellent", *service_range, 7,

        # Food: Rancid, Delicious (Trapezoidal)
        food_rancid = TrapezoidalFuzzySet("Rancid", *food_range, 0, 0, 1, 4)
        food_delicious = TrapezoidalFuzzySet("Delicious", *food_range, 6, 8, 10,

        # Tip: Cheap, Average, Generous (Trapezoidal)
        tip_cheap = TrapezoidalFuzzySet("Cheap", *tip_range, 0, 0, 5, 10)

```

```

tip_average = TrapezoidalFuzzySet("Average", *tip_range, 8, 12, 18, 22)
tip_generous = TrapezoidalFuzzySet("Generous", *tip_range, 18, 22, 25, 2)

# Rule Evaluation
mu_service_poor = service_poor.get_membership(input_service)
mu_food_rancid = food_rancid.get_membership(input_food)
rule1_strength = max(mu_service_poor, mu_food_rancid)

mu_service_good = service_good.get_membership(input_service)
rule2_strength = mu_service_good

mu_service_excellent = service_excellent.get_membership(input_service)
mu_food_delicious = food_delicious.get_membership(input_food)
rule3_strength = max(mu_service_excellent, mu_food_delicious)

# Activation
tip_cheap_activation = np.minimum(rule1_strength, tip_cheap.membership)
tip_average_activation = np.minimum(rule2_strength, tip_average.membership)
tip_generous_activation = np.minimum(rule3_strength, tip_generous.membership)

# Aggregation
aggregated_output = np.maximum(tip_cheap_activation,
                                np.maximum(tip_average_activation, tip_generous_activation))

# Defuzzification
output_tip = defuzzify_centroid(tip_cheap.x, aggregated_output)

return output_tip, {
    'rule1': rule1_strength,
    'rule2': rule2_strength,
    'rule3': rule3_strength
}

```

Tipping Calculation (Gaussian)

This function calculates the tip percentage using **Gaussian** membership functions, which are smooth and differentiable.

```

In [9]: def calculate_tip_gaussian(input_service, input_food):
        """Fuzzy inference system using GAUSSIAN membership functions"""
        service_range = (0, 10)
        food_range = (0, 10)
        tip_range = (0, 25)

        # Service: Poor, Good, Excellent (Gaussian)
        service_poor = GaussianFuzzySet("Poor", *service_range, mean=1.5, sigma=1.5)
        service_good = GaussianFuzzySet("Good", *service_range, mean=5.0, sigma=1.5)
        service_excellent = GaussianFuzzySet("Excellent", *service_range, mean=8.0, sigma=1.5)

        # Food: Rancid, Delicious (Gaussian)
        food_rancid = GaussianFuzzySet("Rancid", *food_range, mean=2.0, sigma=1.5)
        food_delicious = GaussianFuzzySet("Delicious", *food_range, mean=8.0, sigma=1.5)

        # Tip: Cheap, Average, Generous (Gaussian)

```

```

tip_cheap = GaussianFuzzySet("Cheap", *tip_range, mean=5, sigma=3)
tip_average = GaussianFuzzySet("Average", *tip_range, mean=15, sigma=3)
tip_generous = GaussianFuzzySet("Generous", *tip_range, mean=22, sigma=3)

# Rule Evaluation
mu_service_poor = service_poor.get_membership(input_service)
mu_food_rancid = food_rancid.get_membership(input_food)
rule1_strength = max(mu_service_poor, mu_food_rancid)

mu_service_good = service_good.get_membership(input_service)
rule2_strength = mu_service_good

mu_service_excellent = service_excellent.get_membership(input_service)
mu_food_delicious = food_delicious.get_membership(input_food)
rule3_strength = max(mu_service_excellent, mu_food_delicious)

# Activation
tip_cheap_activation = np.minimum(rule1_strength, tip_cheap.membership)
tip_average_activation = np.minimum(rule2_strength, tip_average.membership)
tip_generous_activation = np.minimum(rule3_strength, tip_generous.membership)

# Aggregation
aggregated_output = np.maximum(tip_cheap_activation,
                                np.maximum(tip_average_activation, tip_generous_activation))

# Defuzzification
output_tip = defuzzify_centroid(tip_cheap.x, aggregated_output)

return output_tip, {
    'rule1': rule1_strength,
    'rule2': rule2_strength,
    'rule3': rule3_strength
}

```

Main Execution & Comparison

Finally, we run a series of test cases to compare the results of the three membership functions. The output shows the calculated tip percentage for each method and the activation strength of each rule.

```

In [10]: def main():
    print("=" * 80)
    print("FUZZY LOGIC TIPPING SYSTEM - COMPARING THREE MEMBERSHIP FUNCTIONS")
    print("=" * 80)

    # Test cases: various combinations of food and service ratings
    test_cases = [
        {"service": 1.0, "food": 1.0, "description": "Poor service, Rancid food"},
        {"service": 5.0, "food": 5.0, "description": "Average service, Average food"},
        {"service": 9.0, "food": 9.0, "description": "Excellent service, Delicious food"},
        {"service": 3.0, "food": 8.0, "description": "Poor service, Delicious food"},
        {"service": 8.0, "food": 3.0, "description": "Excellent service, Rancid food"},
        {"service": 6.5, "food": 7.0, "description": "Good service, Good food"}
    ]

```

```

]

for i, test in enumerate(test_cases, 1):
    input_service = test["service"]
    input_food = test["food"]

    print(f"\n{'=' * 80}")
    print(f"TEST CASE {i}: {test['description']}")
    print(f"Input - Service: {input_service:.1f}/10, Food: {input_food:.1f}/10")
    print(f"{'=' * 80}")

    # Calculate using Triangular membership functions
    tip_tri, rules_tri = calculate_tip_triangular(input_service, input_food)

    # Calculate using Trapezoidal membership functions
    tip_trap, rules_trap = calculate_tip_trapezoidal(input_service, input_food)

    # Calculate using Gaussian membership functions
    tip_gauss, rules_gauss = calculate_tip_gaussian(input_service, input_food)

    # Display results
    print("\n--- TRIANGULAR MEMBERSHIP FUNCTIONS ---")
    print(f"Rule Activations: R1={rules_tri['rule1']:.3f}, R2={rules_tri['rule2']:.3f}")
    print(f"Calculated Tip: {tip_tri:.2f}%")

    print("\n--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---")
    print(f"Rule Activations: R1={rules_trap['rule1']:.3f}, R2={rules_trap['rule2']:.3f}")
    print(f"Calculated Tip: {tip_trap:.2f}%")

    print("\n--- GAUSSIAN MEMBERSHIP FUNCTIONS ---")
    print(f"Rule Activations: R1={rules_gauss['rule1']:.3f}, R2={rules_gauss['rule2']:.3f}")
    print(f"Calculated Tip: {tip_gauss:.2f}%")

    print(f"\n COMPARISON:")
    print(f"Triangular: {tip_tri:.2f}%")
    print(f"Trapezoidal: {tip_trap:.2f}%")
    print(f"Gaussian: {tip_gauss:.2f}%")
    print(f"Max Difference: {max(tip_tri, tip_trap, tip_gauss) - min(tip_tri, tip_trap, tip_gauss):.2f}%")

    print("\n" + "=" * 80)
    print("ANALYSIS COMPLETE")
    print("=" * 80)

if __name__ == "__main__":
    main()

```

```

=====
===
FUZZY LOGIC TIPPING SYSTEM - COMPARING THREE MEMBERSHIP FUNCTIONS
=====
===

=====
===
TEST CASE 1: Poor service, Rancid food
Input - Service: 1.0/10, Food: 1.0/10
=====
===

--- TRIANGULAR MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.800, R2=0.200, R3=0.000
Calculated Tip: 7.73%

--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=1.000, R2=0.000, R3=0.000
Calculated Tip: 3.82%

--- GAUSSIAN MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.946, R2=0.029, R3=0.000
Calculated Tip: 5.82%

COMPARISON:
Triangular: 7.73%
Trapezoidal: 3.82%
Gaussian: 5.82%
Max Difference: 3.90%

=====
===
TEST CASE 2: Average service, Average food
Input - Service: 5.0/10, Food: 5.0/10
=====
===

--- TRIANGULAR MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.005, R2=0.990, R3=0.005
Calculated Tip: 14.94%

--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.008, R2=0.987, R3=0.000
Calculated Tip: 14.92%

--- GAUSSIAN MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.136, R2=0.999, R3=0.136
Calculated Tip: 14.05%

COMPARISON:
Triangular: 14.94%
Trapezoidal: 14.92%
Gaussian: 14.05%
Max Difference: 0.89%

```

```

=====
===
TEST CASE 3: Excellent service, Delicious food
Input - Service: 9.0/10, Food: 9.0/10
=====
===

--- TRIANGULAR MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.000, R2=0.200, R3=0.800
Calculated Tip: 20.10%

--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.000, R2=0.005, R3=1.000
Calculated Tip: 22.35%

--- GAUSSIAN MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.000, R2=0.029, R3=0.946
Calculated Tip: 20.82%

COMPARISON:
Triangular:    20.10%
Trapezoidal:   22.35%
Gaussian:      20.82%
Max Difference: 2.25%

=====
===
TEST CASE 4: Poor service, Delicious food
Input - Service: 3.0/10, Food: 8.0/10
=====
===

--- TRIANGULAR MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.400, R2=0.600, R3=0.500
Calculated Tip: 13.96%

--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.667, R2=0.011, R3=0.992
Calculated Tip: 12.85%

--- GAUSSIAN MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.607, R2=0.411, R3=1.000
Calculated Tip: 13.74%

COMPARISON:
Triangular:    13.96%
Trapezoidal:   12.85%
Gaussian:      13.74%
Max Difference: 1.12%

=====
===
TEST CASE 5: Excellent service, Rancid food
Input - Service: 8.0/10, Food: 3.0/10
=====
===

```

--- TRIANGULAR MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.250, R2=0.400, R3=0.600
Calculated Tip: 15.34%

--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.333, R2=0.500, R3=0.500
Calculated Tip: 13.67%

--- GAUSSIAN MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.801, R2=0.135, R3=0.946
Calculated Tip: 13.05%

COMPARISON:
Triangular: 15.34%
Trapezoidal: 13.67%
Gaussian: 13.05%
Max Difference: 2.29%

=====
===
TEST CASE 6: Good service, Good food
Input - Service: 6.5/10, Food: 7.0/10
=====

--- TRIANGULAR MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.000, R2=0.700, R3=0.300
Calculated Tip: 17.00%

--- TRAPEZOIDAL MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.000, R2=1.000, R3=0.500
Calculated Tip: 16.37%

--- GAUSSIAN MEMBERSHIP FUNCTIONS ---
Rule Activations: R1=0.004, R2=0.607, R3=0.801
Calculated Tip: 17.91%

COMPARISON:
Triangular: 17.00%
Trapezoidal: 16.37%
Gaussian: 17.91%
Max Difference: 1.53%

=====
===
ANALYSIS COMPLETE
=====